

## Activity 5 — System Loops: Why the Problem Keeps Coming Back

### Horizon Bank revisited — the fix that made it worse

|                  |                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Topic</b>     | Topic 1                                                                                                          |
| <b>Objective</b> | A3 · K2 — Deduce linkages and patterns to identify key implications on organisational systems.                   |
| <b>Duration</b>  | 50 minutes                                                                                                       |
| <b>Grouping</b>  | Teams of 3-5                                                                                                     |
| <b>Tools</b>     | System Loop ed-tool, ChatGPT / Copilot / Gemini                                                                  |
| <b>Ed-tool</b>   | System Loop Tool — <a href="https://alfredang.github.io/systemloop/">https://alfredang.github.io/systemloop/</a> |

### The Scenario

Return to Horizon Bank Jurong East. Acting on the fishbone, the Regional Director did the obvious thing: he ordered two additional counters staffed at peak, redeploying staff from the back office.

Month 1: average wait fell from 58 to 34 minutes. Everyone celebrated.

Month 4: average wait is back to 61 minutes — WORSE than before. Complaints are at 68 per month. Three more tellers have resigned. Back-office processing is now two days behind, which has started generating a new complaint category of its own.

The Regional Director is baffled: "We added capacity and the problem came back bigger. What am I missing?"

What he is missing is that a branch is not a queue — it is a system of feedback loops. Your team must map them.

### The Evidence

*Horizon Bank — system variables and observed 4-month movements*

| Variable                                     | Month 1  | Month 4 | Direction    |
|----------------------------------------------|----------|---------|--------------|
| <b>Counters staffed at peak</b>              | 3.2      | 5.1     | up           |
| <b>Average wait time (min)</b>               | 58 -> 34 | 61      | down then up |
| <b>Back-office processing backlog (days)</b> | 0.5      | 2.0     | up           |
| <b>Teller errors requiring rework</b>        | 6%       | 14%     | up           |
| <b>Rework returning to the counter</b>       | low      | high    | up           |
| <b>Teller overtime hours / month</b>         | 88       | 196     | up           |
| <b>Staff resignations</b>                    | 2        | 3       | up           |

|                                       |     |     |      |
|---------------------------------------|-----|-----|------|
| (quarter)                             |     |     |      |
| Share of counter staff under 6 months | 44% | 58% | up   |
| Mean transaction handling time (min)  | 9   | 13  | up   |
| Customer complaints / month           | 60  | 68  | up   |
| Digital channel adoption              | 21% | 20% | flat |

## Discussion Questions

1. Map at least two reinforcing (R) and two balancing (B) loops. Which loop did the Regional Director believe he was pulling, and which loop did he actually trigger?
2. Month 1 showed real improvement. Explain the DELAY: why did the harmful loop take three months to overwhelm the helpful one, and what does that mean for how we evaluate quick wins?
3. Follow the back-office redeployment all the way round. Trace how a decision about counters ended up increasing teller resignations.
4. Digital adoption stayed flat at 20% throughout. Which loop is NOT running that should be, and why is a dormant balancing loop as dangerous as an active reinforcing one?
5. Name two leverage points. What makes a leverage point different from simply doing more of the same thing harder?

## Step-by-Step

6. Read the revisited Horizon Bank scenario and the 4-month variable movements.
7. Open the System Loop ed-tool at <https://alfredang.github.io/systemloop/>
8. List the key variables from the table as nodes: counters staffed, wait time, backlog, errors, rework, overtime, resignations, inexperience, handling time, digital adoption.
9. Draw the loop the Director INTENDED: pressure -> counters -> capacity -> wait time down. Label it B1 and confirm it closes as balancing.
10. Now trace the redeployment: where did the back-office staff come from, and what happened downstream? Close that loop and label it R1.
11. Trace the workload consequences on people: overtime, resignations, inexperience, errors. Close it and label it R2.
12. Mark (+) or (-) on every link. Verify each loop: an even number of (-) links makes it reinforcing; an odd number makes it balancing.
13. Identify where the DELAY sits — which loop acts in days and which acts in months.
14. Ask which balancing loop is available but not running. Add B2 (digital adoption) and mark it dormant.
15. Run the GenAI system-thinking prompt from the slide and compare its loops against yours.
16. Circle two leverage points where breaking ONE link would collapse a reinforcing loop.
17. Present: the four loops, why month 1 lied to you, and your two leverage points.

## The GenAI Prompt

Act as a systems thinking facilitator. Help me map the causal feedback loops in this case.

Situation: A bank branch added 2 counters at peak by redeploying back-office staff. Wait time fell from 58 to 34 minutes in month 1, then rose to 61 minutes by month 4 – worse than the starting point.

Observed changes over 4 months: back-office backlog 0.5 -> 2.0 days; teller errors 6% -> 14%; rework returning to the counter rose; overtime 88 -> 196 hrs/month; resignations rose; share of tellers under 6 months 44% -> 58%; mean handling time 9 -> 13 min; digital adoption flat at ~20%.

Produce:

1. At least 2 REINFORCING (R) loops and 2 BALANCING (B) loops. For each, list the variables in causal order with (+) or (-) on each link, and name the loop
2. Explain specifically WHY the month-1 improvement reversed – which loop overwhelmed which
3. Identify the delay in the system and explain why the delay made the wrong decision look right
4. Name 2 leverage points that would break the reinforcing loops, and say what makes them leverage rather than just more effort

## Trainer Debrief

*Expected answers and the points to draw out. Trainer reference.*

### EXPECTED LOOPS:

B1 — CAPACITY CONTROL (balancing, what the Director intended): wait time up (+) -> management pressure (+) -> counters staffed (+) -> service capacity (+) -> wait time DOWN (-). This is the loop that produced the month-1 win. It is real, but it is not the only loop running.

R1 — REDEPLOYMENT BACKFIRE (reinforcing, the loop he triggered without seeing it): counters staffed (+) -> back-office staff removed (+) -> processing backlog (+) -> errors and unresolved cases (+) -> rework returns to the counter (+) -> counter workload (+) -> handling time (+) -> WAIT TIME (+) -> more pressure to staff counters (+). Handling time rising from 9 to 13 minutes is the fingerprint of this loop.

R2 — BURNOUT SPIRAL (reinforcing): workload (+) -> overtime (+) -> stress and burnout (+) -> resignations (+) -> share of inexperienced tellers (+) -> error rate (+) -> rework (+) -> workload (+). Tellers under 6 months rising 44% -> 58% while errors doubled 6% -> 14% is the fingerprint here.

B2 — DIGITAL MIGRATION (balancing, DORMANT): wait time (+) -> willingness to try digital (+) -> digital adoption (+) -> branch transaction volume (-) -> wait time (-). Adoption stayed flat at ~20%, so this loop never engaged. Nothing was done to activate it.

### KEY TEACHING POINTS:

1. THE CENTRAL INSIGHT: he pulled B1 and unknowingly powered R1 and R2. Because R-loops compound, they eventually overwhelm a one-off capacity increase. Adding capacity to a system with an active reinforcing loop buys a temporary win and a bigger relapse.
2. DELAY IS THE TRAP: B1 acts in days (staff a counter, wait drops). R1 and R2 act over months (backlog builds, errors accumulate, people quit). The delay is precisely why the month-1 result validated a decision that was making things worse. Every learner should leave able to say: 'a quick win measured too early is indistinguishable from a mistake.' This is the direct bridge into Topic 3's leading vs lagging indicators.
3. DORMANT LOOPS: B2 was available the whole time and nobody activated it. Shifting simple transactions to digital reduces the load that feeds R1. Asking 'which balancing loop is not running?' is often more productive than adding effort to loops that are.
4. LEVERAGE vs EFFORT: more counters = effort (you must sustain it forever, and it feeds R1). Leverage = breaking a link in a reinforcing loop: triage complex cases away from the counter; eliminate the triple data entry so error rate falls; protect back-office capacity so rework never returns to the counter; activate B2 with in-branch digital ambassadors. Leverage changes the structure; effort fights the structure.
5. This is why Topic 1 ends here. 5 Whys gave depth, Fishbone gave breadth, Pareto gave priority — and System Loops explains why well-intentioned fixes rebound. Learners now have the full diagnostic set before touching solutions in Topic 2.

### **Self-Check — Is Your Output Finished?**

Your loop map is valid when: every loop CLOSES (the last variable feeds back to the first); every link carries a (+) or (-); each loop is correctly typed as R or B by counting negative links; you can explain the month-1-to-month-4 reversal by naming which loop overwhelmed which; and your leverage points break a LINK in a reinforcing loop rather than adding more effort to a balancing one. If your answer is 'hire more people', you have described effort, not leverage.